CREATING A CLOUD VM IN AWS

Step-by-Step Guide

This guide walks you step by step through creating your first virtual machine (a "Cloud VM") on Amazon Web Services (AWS), using the AWS free tier. It links directly to concepts from your EXIN Cloud Computing Foundation course — in particular Infrastructure as a Service (IaaS), on-demand self-service, and rapid elasticity.

Prerequisites

- An AWS account (sign up free at aws.amazon.com — the Free Tier covers this exercise at no cost for 12 months on eligible services).
- A computer with internet access and a terminal / command prompt (for connecting to your VM afterwards).
- About 30–45 minutes.

Step 1 — Sign In to the AWS Management Console

1. Go to https://console.aws.amazon.com/ec2/ and sign in with your AWS account.
2. Once signed in, you land on the EC2 (Elastic Compute Cloud) Dashboard — this is the service that lets you create and manage virtual machines ("instances") in AWS.

> *Note: EC2 instances are AWS's implementation of IaaS — you rent raw virtual compute capacity and manage the operating system yourself, which is exactly the IaaS model described in the EXIN course.*

Step 2 — Create a Key Pair

A key pair is how you securely log in to your VM later — think of it as a digital key, instead of a password.

1. In the left navigation pane, under Network & Security, choose Key Pairs.
2. Click Create key pair (top right).
3. Enter a name, e.g. peter-vm-key.
4. Key pair type: choose ED25519 (or RSA if ED25519 is not available).
5. Private key file format: choose .pem (for Mac/Linux/PowerShell) or .ppk (if you will use PuTTY on Windows).
6. Click Create key pair — the private key file downloads automatically. Save it somewhere safe; you cannot download it again.

Step 3 — Launch Your VM (EC2 Instance)

1. From the EC2 Dashboard, click Launch instance.
2. Name and tags: give your instance a name, e.g. Peter-Test-VM.
3. Application and OS Images (AMI): keep the default Amazon Linux 2023 AMI — it is marked "Free tier eligible".
4. Instance type: keep the default free-tier eligible type (e.g. t2.micro / t3.micro).
5. Key pair (login): select the key pair you created in Step 2.
6. Network settings: keep the default VPC and subnet. Under Firewall (security groups), keep "Allow SSH traffic from Anywhere (0.0.0.0/0)" checked so you can connect afterwards.
7. Configure storage: keep the default root volume settings.
8. Review the Summary panel on the right, then click Launch instance.
9. You'll see a Success banner — your VM is now being created.

Step 4 — Connect to Your VM

1. From the left navigation pane, choose Instances, and locate your instance by name.
2. Select the instance, then click Connect (top right).
3. Open the SSH client tab — AWS shows you a ready-made command similar to:

```
ssh -i "peter-vm-key.pem" ec2-user@ec2-XX-XXX-XXX-XXX.compute.amazonaws.com
```

4. Open a terminal on your own computer, navigate to the folder containing your downloaded key file, and (on Mac/Linux) run:

```
chmod 400 peter-vm-key.pem
```

5. Paste and run the SSH command from AWS. When asked "Are you sure you want to continue connecting?", type yes and press Enter.
6. You are now logged in to your own Cloud VM running in AWS.

Step 5 — Stop the Instance (Manage Costs)

Even on the Free Tier, it's good practice to stop instances you're not using.

1. Go back to Instances, select your VM's checkbox.
2. Click Instance state (top right), then Stop instance.
3. Confirm by clicking Stop. The state changes from Stopping to Stopped.

> *Note: Stopping an instance is different from terminating it: Stop keeps your VM and its data so you can start it again later; Terminate deletes it permanently.*

How This Connects to Your Course

- On-demand self-service: you provisioned a VM yourself, in minutes, with no human intervention from AWS.
- Rapid elasticity: you could easily launch more instances, or a larger instance type, in the same way.
- Resource pooling: your VM runs on shared AWS hardware, isolated from other customers.
- Measured service: AWS meters your usage (compute hours, storage) — this is why stopping unused instances matters.